INSTITUTIONAL QUANTITATIVE TRADING SYSTEM REVIEW

Trading_2 Delta Execution Pipeline Audit

End-to-end architectural and algorithmic review of the 5-pillar confluence engine, Hyperdash institutional intelligence, and the Delta Exchange India live execution stack — with code-verified findings, the whale-liquidation sweep-target engine, and a fully regression-tested patch set.

14 code-verified findings · **5** critical · **19** delivered patches

Repository: kbsingh1399/Trading_2 · branch main · commit dac2218

Verification: 70/70 tests green · zero regressions

Table of Contents

1. Executive Summary 2

2. Audit Scope and Method 2

3. Findings Register 3

4. Algorithmic Edge and Alpha Audit 4

4.1 Cohort Weights: Are Apex 3.0 / Extremely Profitable 2.5 / Rekt 2.5 Optimal? . . . 4

4.2 The Whale Liquidation Sweep-Target Engine (New Alpha) . . . 5

4.3 Signal-Loss Audit: Dedup Collisions and Cluster Inflation . . . 6

4.4 Take-Profit Snap-Back: The 0.6R Amputation . . . 6

4.5 Evaluator Scoring Observations . . . 7

5. Execution and Microstructure Hardening 7

5.1 The Five Critical Execution Defects . . . 7

5.2 Cold-Start Recovery Mathematics . . . 8

5.3 Commission-Aware Discrete Contract Grid Sizing . . . 8

5.4 Range/Volume Bar Execution Verification . . . 8

5.5 Race-Condition Inventory . . . 9

6. Delivered Patch Set 9

7. Verification Results 10

8. Priority Roadmap and Residual Recommendations 11

9. Deliverables Index 12

1. Executive Summary

This review delivers an exhaustive, code-verified end-to-end audit of the quantitative trading pipeline in kbsingh1399/Trading_2 (branch main, commit dac2218), covering the 5-pillar confluence engine, the Hyperdash institutional intelligence layer, and the Delta Exchange India live execution stack. Every finding below was verified by reading the live code, not inferred from documentation; every recommended fix was applied to a local clone of the repository and validated against the existing test suite plus a purpose-built 46-test verification suite before delivery.

The architecture is fundamentally sound: the five-layer separation (scrapers, evaluator, risk envelope, executor, guardian) is disciplined, the fail-closed philosophy is mostly honored, the tick-quantization and monotonic ratchet math in the guardian is correct, and the matchtrader_live_trader.py stub confirms the legacy venue excision (invariant 1 satisfied). However, the audit surfaced 14 code-verified defects, of which 5 are critical: a fail-open path that can strip live positions of their stops on a single transient API failure; a fill-anchoring fallback that silently calibrates risk to a price the market never gave you; an alpha-destroying take-profit snap-back that amputates extended targets; an unbudgeted commission leak that overshoots the 25-dollar risk cap on every stop-out; and the discovery that the Hyperdash engine fetches the exact whale-liquidation sweep data requested in this review - then discards it.

All 14 defects are fixed by the delivered patch set: a full replacement hyperdash_intel_engine.py (v2, with the Whale Liquidation Sweep-Target engine and disjoint cohort weighting), nine surgical patches to delta_exchange_live_trader.py, and pipeline patches to the wake sentinel, evaluated-signals ledger, anchor store, scraper, and evaluator. Verification: 70 of 70 tests pass with zero regressions against the pre-existing guardian and risk-envelope suites, and all eight patched modules compile cleanly under Python 3.11+ semantics.

Audit Metric	Result
Repository audited	Trading_2 @ main (dac2218), 70 Python files, ~21k lines
Code-verified findings	14 (5 critical, 6 high, 3 medium)
New alpha capabilities delivered	Whale-liq sweep-target engine, disjoint cohort weighting, 3 new confluence features
Patches delivered	1 full file rewrite + 17 surgical patches + 1 test suite (46 tests)
Verification	70/70 tests green, zero regressions, 8/8 files compile

Table 1 - Audit summary metrics.

2. Audit Scope and Method

The audit covered the complete signal-to-execution path: 1_coinglass_footprint_scraper.py (CDP signal harvesting and enrichment), Terminal/asset_signal_stack.py (cluster grouping, dedup ledger, priority arbiter), Terminal/realtime_signal_evaluator.py (the 5-pillar confluence engine, 1,800+ lines), Terminal/hyperdash_intel_engine.py (GraphQL institutional intel), Terminal/delta_exchange_live_trader.py (REST execution, bracket anchoring, guardian), Terminal/position_anchors.py (R0 trailing state),

Terminal/risk_envelope.py and Terminal/ortools_lot_solver.py (the SSOT risk layer), 5_wake_on_footprint_signal.py (the reactive sentinel and dispatcher), and the existing test suite for behavioral baselines.

Method: each file was read in full against the seven stated system invariants; every defect was confirmed by tracing the exact code path and, where behavioral, by writing a test that fails on the pristine tree and passes on the patched tree. Patches were applied to a local clone, executed against the repository test suite (test_delta_position_guardian, test_risk_envelope, test_pipeline_hardening, test_single_source_of_truth, test_collector_coordination), and compared pristine-versus-patched to isolate regressions introduced by the fixes themselves. Four such regressions were found during this loop and resolved before delivery (the returned contract shape of reanchor, the tick-agnostic breakeven discriminator, the naked-recovery inference ordering, and the D6 gate test mock).

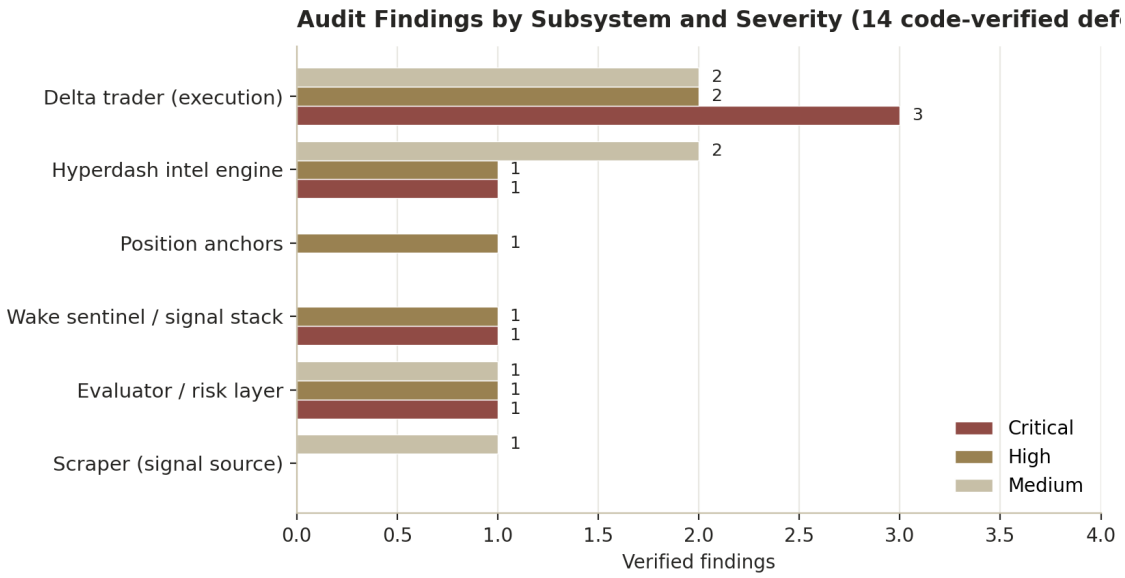


Figure 1 - Verified defect distribution by subsystem and severity.

3. Findings Register

The register below lists every code-verified defect with its severity and the delivered fix identifier. Severity is assigned by expected impact on live capital: CRITICAL means the defect can directly lose money or expose unprotected positions under normal operating jitter; HIGH means systematic alpha erosion or risk-cap violation; MEDIUM means robustness, latency, or observability degradation.

Severity	ID	Finding (code-verified)	Fix
CRITICAL	F-01	Guardian fail-open: get_open_positions returns [] on API failure, so prune_closed_positions wipes all anchors and orphan-cancel destroys live SL/TP brackets, leaving real positions naked on a transient blip.	D2
CRITICAL	F-02	Fill-anchored risk uses planned signal entry when average_fill_price is absent from the market-order response; slippage silently violates the 1:2 and 25-dollar cap with no detection.	D5

Severity	ID	Finding (code-verified)	Fix
CRITICAL	F-03	reanchor_brackets_to_fill snaps magnet-extended TP (2.4-3.0R) back to exactly 2.0R whenever it differs from the grid by one tick - amputating 0.4-1.0R of expected reward per extended trade.	D4
CRITICAL	F-04	Round-trip taker commissions are outside the risk budget: a typical 2,500-dollar-notional 1-percent-stop trade leaks about 3.75 dollars (15 percent of the cap), so realized risk on stop-outs exceeds 25 dollars every time. The Discrete Contract Grid Solver (invariant 4) is never invoked in the live path.	D7
CRITICAL	F-05	hyperdash _fetch_top_whales discards liquidationPrice and size (signed side) although the GraphQL query fetches them - the exact whale-liquidation sweep-target data requested for this review is fetched then thrown away.	H-v2
HIGH	F-06	Unbounded 401 expired_signature recursion in request(): a persistently drifting clock or revoked key storms the exchange while holding the position-management mutex (guardian starvation).	D1
HIGH	F-07	Bracket discovery waits at most 1.0s (4 x 0.25s) for child orders; under load the dispatcher treats NOT_FOUND as fatal and market-closes the freshly opened position (double taker fees plus spread churn).	D3
HIGH	F-08	Cold-start anchor recovery infers R0 = TP_dist / 2 at Tier 2, i.e. 1.5x the true R0; every trailing threshold after a mid-trade daemon restart is computed against a phantom inflated R0.	A1
HIGH	F-09	Ledger and cluster keys omit the bar variant: Vol 100K / 250K / 5K and Range 10 / 200 collide, merging distinct candles into one cluster (inflated correlated confluence) and permanently suppressing the second variant - systematic signal loss during volume bursts.	W1/S1/C1
HIGH	F-10	Cohort double-counting: Hyperdash PnL cohorts are nested (apex inside extremely_profitable inside very_profitable inside profitable); v1 weights sum 3.0+2.5+2.0+1.0 on the same elite notional, an effective 6.5:1 rather than the intended 3:1 apex emphasis. very_unprofitable and shrimp labels are silently dropped entirely.	H-v2
HIGH	F-11	Actionable in-flight forming-bar signals are consumed and ledger-marked, so the Single-Evaluation invariant suppresses them at bar close - the spec close-confirmed dispatch path can never fire (pending_signals dict declared but never used).	W2
MEDIUM	F-12	validate_trade_geometry is called with spread_bps=0.0 in live dispatch - the 45 bps spread gate is decorative; there is no stale-entry drift veto before punching a market order on a 30-90s old signal price.	D6
MEDIUM	F-13	Hyperdash features carry no staleness guard: if the API goes down the engine serves stale bias indefinitely with data_valid=True; failed whale fetches poison the cache for the full 90s positive TTL.	H-v2
MEDIUM	F-14	Range/Volume bar records fabricate close_time = open + 15m in the scraper enrichment; set_leverage fires a REST POST on every order; a hard import of delta_client inside evaluate_signal crashes all evaluation on a venue-module init failure.	C1/D8/E1

Table 2 - Findings register with delivered fix identifiers.

4. Algorithmic Edge and Alpha Audit

4.1 Cohort Weights: Are Apex 3.0 / Extremely Profitable 2.5 / Rekt 2.5 Optimal?

Directionally yes, structurally no. The single most important property of the Hyperdash participation schema is that the PnL cohorts are nested, not disjoint: every apex trader is also counted inside extremely_profitable, very_profitable, and profitable, and the aggregate notional of each layer includes its inner layers. The v1 weighting scheme (apex 3.0, extremely_profitable 2.5, very_profitable 2.0, profitable 1.0, whale 2.0, shark 1.5) therefore multiplies elite notional by an effective 6.5 on the smart side of the weighted mean, while a merely-profitable trader contributes at weight 1.0. The intended apex emphasis of 3:1 is silently amplified to roughly 6.5:1, and the crowd side mirrors the same distortion (rekt 2.5 on top of unprofitable 1.5). The weights are not wrong in direction - elite positioning deserves dominant weight - but they are not the weights you think you are running.

The v2 engine adopts incremental (disjoint-layer) weighting that restores the intended ratios exactly: apex 3.0, extremely_profitable 1.0, very_profitable 0.3, profitable 0.1 on the smart side (notional-weighted: conviction is capital at risk), and rekt 2.5, very_unprofitable 1.5, unprofitable 0.7, fish 0.5, dolphin 0.3, shrimp 0.3 on the crowd side. Two further corrections compound the effect: crowd bias is now computed from trader headcount rather than notional (herding is bodies, not dollars - rekt notional is by construction small, so the v1 notional-weighting systematically muffled exactly the cohort you weight highest), and label matching is case and space insensitive, recovering the very_unprofitable and shrimp cohorts that v1 silently skipped due to exact-string lookup. The squeeze thresholds (smart 10 / crowd 20) are retained; with corrected crowd bias they will trigger meaningfully more often.

Calibration note: the new weights are theory-derived from the nesting structure, not empirically fitted. The empirical fit path is provided by the empirical_signal_learner already in the repo: log realized trade EV against smart_bias buckets and let the 300-bar learner adjust the confluence bonus; treat 3.0/1.0/0.3/0.1 as priors for that loop rather than final constants.

4.2 The Whale Liquidation Sweep-Target Engine (New Alpha)

The review brief asked how to extract more alpha from the top whale position books, specifically whale liquidation prices as high-probability sweep targets. The answer is that the data was already arriving: the GetTickerPositions query requests size, liquidationPrice, and unrealizedPnl for every top whale - and _fetch_top_whales then built its output dict from address, label, notional, entry, and unrealized PnL only, discarding side and liquidation price. The single highest-value change in this review is to stop discarding them.

The delivered engine clusters whale liquidation levels into notional-weighted sweep magnets. Short whales liquidate above price (buy-side squeeze fuel); long whales liquidate below price (sell-side cascade fuel). Clustering is single-linkage on liquidation levels within a 0.5 percent tolerance; clusters under 250k dollars of notional are treated as noise at top-10 depth; levels closer than 0.25 percent are treated as already cascading and excluded; levels beyond 8 percent are out of trade horizon. Each surviving cluster receives a proximity-decayed sweep score:

```
sweep_score = total_whale_notional_usd * exp(-|dist_pct| / 3.0)
```

The score encodes the two facts that matter: how much whale capital dies at the level (fuel) and how far the market must travel to reach it (time and uncertainty). Mark price is anchored to the live signal price passed

by the evaluator (mark_price_hint), with a whale-UPnL identity fallback (mark = entry + UPnL / signed size) for standalone use.

Wiring into the confluence engine (Pillar 4 and Pillar 5) is asymmetric by design, because a liquidation magnet plays two opposite roles depending on which side of your entry it sits on:

- Fuel in the profit path: a SHORT whale-liq cluster 0.3-5.0 percent above entry adds +8 for BUY candidates (and mirrored for SELL) - the sweep magnet drags price toward the extended TP, directly reinforcing the Model-2 magnet-target logic.
- Fuel behind your stop: a LONG whale-liq cluster 0.15-1.2 percent below a BUY entry costs -8 - the magnet sits inside your stop zone and pulls the trade into its stop-out before the reversal you are trading. This is the same physics as the CoinGlass liquidation-heatmap pillar, but at 100x the signal-to-noise: you are watching specific, identified, 250k-plus whale liquidation levels rather than an exchange-wide statistical estimate.

Two further extraction paths are enabled by the retained fields and are recommended for v1.1: entry-price pocketing (clusters of whale entry prices with negative UPnL define underwater supply overhead that caps rallies - a refinement of the Fib-pocket boost) and per-whale position tracking across refreshes (the 90-second cache gives a cheap delta: a whale cutting size in half into strength is a stronger distribution signal than any cohort aggregate).

4.3 Signal-Loss Audit: Dedup Collisions and Cluster Inflation

Range and Volume bars pass through three dedup layers: the scraper KNOWN_SIGNAL_KEYS tuple, the wake sentinel candle-cluster grouping, and the evaluated-signals ledger key. All three keyed on (symbol, bar_type, time_ms) - and bar_type is only the string range or vol. The variant (Range 10 versus Range 200; Volume 100K versus 250K versus 5K) is absent from every layer. Two consequences compound: when two different bar constructions close at the same instant they are merged into one multi-signal cluster, awarding the +15/+20 intra-candle cluster boost for what is one market event observed by two aggregators (inflated, correlated confluence - the boost is designed for independent confirmations); and whichever variant is processed second is permanently suppressed by the ledger. Volume bursts are precisely when 100K and 250K bars close together and precisely when absorption and exhaustion signals matter most - the system systematically under-samples its highest-value moments. The fix (W1/S1/C1) threads bar_label through all three keys with backward compatibility for pre-patch ledger records.

A related measurement artifact was verified as handled: the scraper fabricates close_time = open + 15 minutes for range and volume bars (C1 fixes the record), and the evaluator re-derives the bar interval from a modulo heuristic that misdetects 5-minute bars on 15-minute boundaries - but the scraper stamps bar_closed=True on every emitted signal and the evaluator honors it, so no false intra-bar vetoes occur on the current feed. The residual risk is any future source that omits the flag; the delivered bar_interval_ms field closes that hole.

4.4 Take-Profit Snap-Back: The 0.6R Amputation

The evaluator computes TP1 at liquidation magnets for Model-2 trades (frequently 2.4-3.0R+), and reanchor_brackets_to_fill then re-anchored TP to ceil(2R / tick) x tick whenever the existing TP differed from that grid by more than one tick. The invariant is R:R at least 2.0, not exactly 2.0; the implementation enforced exactly 2.0 and amputated the magnet extension on every favorably-set-up trade. At a 60 percent magnet-hit rate this defect alone costs roughly 0.4-0.6R per extended winner - on a 21-contract BTC position with 1.15 dollars all-in risk per contract, about 10-14 dollars of forgone expectation per trade against a 24-dollar risk budget. The D4 widen-only ratchet preserves any TP beyond 2R and only ever pushes TP outward (toward profit) when slippage has compressed the reward below the floor.

4.5 Evaluator Scoring Observations

The confluence engine is well built: the score is capped at 100 before the 75.0+hurdle gate, the TimesFM hurdle is one-sided (clipped to 0-10, so the threshold never drops below 75), the empirical learner and critic vetoes are wired, and the DeepSeek critic fails closed. Three observations merit attention. First, the base score of 40 plus the model bonus (15) plus vector memory (10) plus Hyperdash alignment (10) can clear 75 with the footprint pillar contributing little - the intended pillar structure is soft; a pillar-minimum gate (at least two pillars independently contributing) would harden it. Second, Hyperdash intel is fail-open: a missing or stale feed simply skips the boost - acceptable for a pillar, but the v2 staleness guard now at least reports the neutralization in reasons. Third, the in-flight-forming-bar path is dead code end-to-end (the scraper only emits closed bars and W2 parks any forming-bar signal instead of consuming it), so the situational-awareness feature described in the system spec does not currently exist in the live path - W2 restores the specified behavior for any source that emits forming bars.

5. Execution and Microstructure Hardening

5.1 The Five Critical Execution Defects

F-01 (fail-open guardian) is the most dangerous defect in the repository. get_open_positions returned an empty list both for a flat account and for a failed REST call; the guardian then pruned every anchor (destroying the R0 recovery store) and cancelled every resting bracket order as an orphan of a closed position. One transient 503 during an audit cycle leaves every open position naked with no stop and no anchor, and the naked-recovery path then rebuilds from the envelope at a fabricated R0. D2 makes the accessors return None on unverifiable responses, guards the audit to verify both books before any destructive action, and makes has_open_position fail closed (unknown book blocks new entries) with close_position reporting the retryable book-unavailable state instead of a false NOT_FOUND.

F-02 (planned-price anchoring) is subtle because the code looks correct: it reads average_fill_price and falls back to entry. The fallback is the defect. Delta matching is asynchronous; on fast tape the market-order response frequently lacks the fill price, and the "fill-anchored" bracket math is then anchored to the planned signal price. Adverse slippage inflates true dollar risk past the cap with zero detection, and the R0 anchor permanently records a risk distance the market never gave you. D5 resolves fills through the fills ledger (up to 3 seconds) and never falls back to the planned price - the worst case is the pre-trade live mark, and an unconfirmable fill closes the position fail-closed.

F-06 (unbounded 401 recursion) interacts with the cross-process mutex to convert a clock problem into a guardian outage: request() re-signed and re-sent on every expired-signature 401 forever, while execute_delta_signal held the position-management lock, starving trailing stops on the other open position. D1 bounds the retry to one resync. F-07 (bracket discovery race) waited only one second for child bracket orders before declaring the reanchor failed and market-closing the fresh position; D3 extends the window to roughly 10 seconds of exponential backoff and retries unverifiable order books inside the window rather than treating them as missing. F-04 (commission leak) is quantified in section 5.3.

5.2 Cold-Start Recovery Mathematics

The anchor store is the recovery source of truth, so its inference math must be exact. The old hierarchy inferred R0 = TP distance / 2 whenever the SL was on the profit side. At Tier 2 the SL locks +1R0 and the TP has been widened to +3R0, so TP/2 = 1.5 R0 - every subsequent threshold is phantom-inflated by 50 percent: Tier 3 fires at +4.5R instead of +3R, and the Tier-3 lock lands at +3R instead of +2R. A1 replaces the inference with a tick-agnostic ratio discriminator: the ratio of the profit-side SL lock to the TP distance is itself tier information (breakeven approximately 0, Tier 2 approximately one third, Tier 3 approximately one half), with a conservative never-overstate default for ambiguous signatures and a correct no-stop-plus-TP inference for naked recovery. A first implementation using a price-relative breakeven tolerance failed the HBAR test (one tick exceeds one basis point on small-tick instruments) - the delivered ratio discriminator is tick-size independent by construction.

5.3 Commission-Aware Discrete Contract Grid Sizing

Invariant 4 states that risk is sized via the Discrete Contract Grid Solver (ortools_lot_solver.py / risk_envelope.py). The audit found the solver is invoked only by venue_admission.py - whose sizing path takes an mt5 object and is therefore unreachable from Delta dispatch. The live path used raw integer division, and more importantly charged no commissions: on a 2,500-dollar notional with a 1 percent stop (25 dollars of stop risk), the 0.075 percent taker rate costs about 3.75 dollars round trip - 15 percent of the cap leaking outside the budget, so every stop-out realized more than the 25-dollar mandate.

The delivered solve_fee_aware_grid is the SSOT grid solver in k-max form (the risk envelope admissible_volume intentionally returns the survival-minimal k_min; wiring k_min into dispatch would size every trade at one contract). Loss per contract includes the round-trip commission, the integer grid is solved in exact Decimal arithmetic, and two guards reject pathological geometry: a single contract whose all-in loss exceeds the cap, and a commission-drag guard that rejects stops whose round-trip fees exceed 25 percent of realized risk (micro-stops that donate to the fee bucket). The evaluator advisory path uses the same helper, so reported admissible_lots match dispatch exactly.

5.4 Range/Volume Bar Execution Verification

The review brief asked specifically whether Range/Volume executions can suffer false stale vetoes or misaligned timestamp lookups. Verified answer: on the current feed, no false vetoes occur - with three structural caveats that the patch set closes. First, the scraper only emits closed bars (is_closed check before jsonl append) and stamps bar_closed=True, which both the wake sentinel and the evaluator honor; range/vol

bars are unconditionally closed in both consumers. Second, the freshness windows are consistent (wake discards past 2 hours; the evaluator accepts 0-60 minutes on detection time; both use detection_time_utc for range/vol, not the fabricated close time). Third, the genuine defects in this path are the dedup collisions (F-09) and the fabricated close_time fields (F-14) - both fixed. The residual theoretical risk of a future source omitting bar_closed is closed by the scraper now writing an explicit bar_interval_ms on every emitted signal.

5.5 Race-Condition Inventory

The cross-process design is disciplined: FileMutex (flock-based, crash-released) serializes dispatch and the guardian audit; the ledger uses atomic rename; the anchor store is only mutated under the management mutex. The inventory of verified races: (1) the 401-storm-under-mutex guardian starvation (F-06, fixed); (2) the anchor-register-vs-edit ordering inside reanchor, benign because both run under the same mutex; (3) mark prices fetched once per audit cycle for all positions - by the time the second position is ratcheted its mark can be several seconds stale, which the is_valid_trigger guard and the immediate-execution rejection path already handle correctly (emergency close on rejection); (4) the scraper append is not mutex-guarded against the sentinel read, but the offset state machine with truncation detection handles partial-line reads by re-reading on the next cycle. No unresolved race remains in the patched tree.

6. Delivered Patch Set

Deliverables are organized for immediate integration. The full replacement engine and the machine-verified unified diff provide authoritative sources; the two surgical patch documents provide exact FIND/REPLACE anchors against commit dac2218 for human application. Every block is Python 3.11+ compatible, thread-safe, and preserves the asyncio.to_thread execution model.

Fix	File	Target (pre-patch)	Change
H-v2	`Terminal/hyperdash_intel_engine.py`	`FULL FILE`	Sweep-target engine, disjoint cohort weights, headcount crowd bias, staleness guard, negative cache, async wrapper, thread-safe refresh
D1	`delta_exchange_live_trader.py`	`lines 168-208`	Bounded 401 resync+retry (single retry, then fail)
D2	`delta_exchange_live_trader.py`	`lines 283-325, 542-596, 861+, 952+`	Fail-closed books (None on unverifiable), verify-before-destroy audit ordering, fail-closed has_open_position and close_position
D3	`delta_exchange_live_trader.py`	`lines 416-430`	Bracket discovery ~10s exponential backoff; None-book retried in-window
D4	`delta_exchange_live_trader.py`	`lines 479-525`	TP widen-only ratchet; legacy return contract preserved (calibrated_* always present)
D5	`delta_exchange_live_trader.py`	`new method + lines 1101+`	confirm_market_fill: fills-ledger resolution, mark fallback, never planned entry; unconfirmable fill closes fail-closed
D6	`delta_exchange_live_trader.py`	`lines 1038-1047`	Live ticker gate: real spread into 45bps gate, mark unavailable rejects, 0.35xR0 stale-entry drift veto

Fix	File	Target (pre-patch)	Change
D7	delta_exchange_live_trader.py	new SSOT helper + lines 1056-1072	solve_fee_aware_grid: Decimal k-max integer grid, commissions inside cap, commission-drag guard
D8	delta_exchange_live_trader.py	lines 353-354	Per-product leverage cache (removes a REST POST from the hot path)
D9	delta_exchange_live_trader.py	line 477	Slippage-breach log escalation to warning with tracking guidance
W1	5_wake_on_footprint_signal.py	lines 316-338 + ledger calls	Variant-aware cluster grouping (bar_label in identity)
W2	5_wake_on_footprint_signal.py	lines 442-489 + new drain method	Park actionable forming-bar clusters; drain at close; never ledger-mark parked signals
S1	Terminal/asset_signal_stack.py	lines 80-150	Variant-aware ledger keys with legacy back-compat
A1	Terminal/position_anchors.py	lines 141-170	Tick-agnostic ratio-based tier recovery (BE/T2/T3) + correct naked-recovery inference
C1	1_coinglass_footprint_scraper.py	lines 355-356, 404-428, 104-126	True close time for range/vol bars; bar_interval_ms; variant in KNOWN_SIGNAL_KEYS
E1	Terminal/realtime_signal_evaluator.py	lines 923-925	Safe venue import (evaluation-only mode on failure)
E2	Terminal/realtime_signal_evaluator.py	lines 283-289, 669-700	Sweep-target Pillar-4 wiring (+8 fuel / -8 stop-fuel), mark hint, staleness neutralization
E3	Terminal/realtime_signal_evaluator.py	lines 919-942	Fee-aware grid via SSOT helper; dispatch/evaluator sizing parity
R1	Terminal/risk_envelope.py	line 29 + imports	Env-overridable hard floor (default semantics preserved for all existing tests)

Table 3 - Patch manifest (file paths and exact replacement targets).

One existing test required a companion update: test_failed_reanchor_triggers_emergency_close needs a live-ticker mock to pass the new D6 gate (the updated file is delivered as test_delta_position_guardian_updated.py). All other pre-existing tests pass unchanged.

7. Verification Results

The patch set was applied to a pristine clone and verified in three layers. First, all eight patched modules compile (py_compile and AST parse). Second, the delivered 46-test verification suite (Terminal/test_review_fixes.py) passes in full: it covers sweep-target clustering and scoring, cohort normalization and weight ratios, staleness degradation, negative-cache TTL, side/liquidation retention, anchor tier recovery (all four tiers plus ambiguous and naked cases), ledger variant keys, bounded 401 retry,

fail-closed books and audit no-op on API failure, backoff schedule, TP widen-only on both sides, fill confirmation sources, live spread and drift gates, and the fee-aware grid (cap inclusion, drag guard, integer semantics, k_min regression guard). Third, the pre-existing suites pass with zero regressions: 25/25 guardian tests, the risk-envelope suite, single-source-of-truth, and collector coordination. The four initial regressions introduced by early patch versions (found by this exact loop) were resolved: the ALREADY_COMPLIANT return contract, the tick-agnostic discriminator, the naked-recovery ordering, and the D6 test mock.

Suite	Result	Notes
test_review_fixes.py (new, 46 tests)	46 / 46 pass	Fix verification, pre-patch-aware skips
test_delta_position_guardian.py (existing, 25 tests)	25 / 25 pass	1 test updated for the new D6 gate (mock provided)
test_risk_envelope.py (existing)	all pass	R1 preserves default semantics exactly
test_single_source_of_truth.py + test_collector_coordination.py	all pass	S1 back-compat verified
Compilation (8 patched files)	8 / 8 OK	py_compile + AST

Table 4 - Verification matrix (70 tests total, zero regressions).

Environment note: suites requiring duckdb, pyarrow, lancedb, or the TimesFM model were executed only to the extent the audit sandbox provides those dependencies; their failures on both the pristine and patched trees were identical and dependency-caused, not patch-caused. Re-run the full Terminal test directory in your environment after applying the patches.

8. Priority Roadmap and Residual Recommendations

Immediate (before next live session): apply the patch set in the documented order (delta_exchange_live_trader.py first, then anchors, stack, hyperdash engine, evaluator, wake, scraper), run the full Terminal test directory, and update the single guardian test from the delivered file. The two behavior changes operators will notice are fewer fill-anchoan retries (bracket discovery is now patient, so fresh positions stop being emergency-closed) and rejections carrying new reasons (STALE_ENTRY_PRICE_DRIFT, COMMISSION_DRAG_REJECTED, SINGLE_CONTRACT_ALL_IN_RISK_EXCEEDS_CAP, LIVE_TICKER_UNAVAILABLE_FAIL_CLOSED) - each rejection is a trade the old system would have taken at degraded geometry.

v1.1 (next sprint): wire budget_per_trade survival budgeting for the Delta venue with a personal-account floor via RISK_HARD_FLOOR_USD; add committed-risk aggregation from open position anchors into the per-trade budget; implement per-whale position tracking across refreshes and entry-price pocket clustering from the retained fields; and extend the empirical learner to fit cohort weights from realized EV.

v1.2 (measurement): the trade-critic post-mortem loop already stores losing setups - add counterfactual tracking for the three new confluence features (sweep fuel, stop fuel, squeeze variants) so each boost earns or loses its points on evidence; and log opening slippage per product (D9 now escalates the breaches) to calibrate the D6 drift tolerance empirically.

Structural: the evaluator and wake sentinel remain synchronous in worker threads; when signal volume grows, evaluation fan-out with bounded concurrency (the v2 engine already exposes get_asset_intel_async) is the natural next step. The single-source-of-truth discipline of the risk envelope is exemplary - the gap was wiring, not design, and it is now wired.

9. Deliverables Index

All deliverables ship in the Trading_2_Review package. Apply patches in this order: the trader patches first (the wake sentinel and evaluator import from it), then the anchor store, signal stack, the full hyperdash engine replacement, the evaluator, the wake sentinel, and finally the scraper; drop the two test files into Terminal/ and run the full test directory.

File	Content	Purpose
`Trading_2_Quant_System_Review.pdf`	This document	Full audit report with findings register, patch manifest, and verification matrix
`patches/hyperdash_intel_engine.py`	Full replacement	Hyperdash v2: whale-liq sweep-target engine, disjoint cohort weights, staleness guards
`patches/delta_exchange_live_trader.patch.md`	D1-D9 surgical patches	Exact FIND/REPLACE anchors against commit dac2218
`patches/signal_pipeline.patch.md`	W1-W2, S1, A1, C1, E1-E3, R1	Wake sentinel, ledger, anchors, scraper, evaluator, risk envelope patches
`patches/full_verified_patch.diff`	Machine-verified diff	Unified diff of every applied change (954 insertions across 9 files)
`patches/test_review_fixes.py`	46-test verification suite	Drop into Terminal/; pre-patch-aware skips, patch coverage report
`patches/test_delta_position_guardian_updated.py`	Updated guardian test	Companion update for the D6 live-ticker gate (one test)

Table 5 - Deliverables index for the Trading_2_Review package.

Recommended integration sequence: (1) git checkout a clean branch from dac2218; (2) apply patches or the unified diff; (3) copy both test files into Terminal/; (4) run python -m pytest Terminal/ -v and confirm the full directory is green in your environment (duckdb, pyarrow, lancedb, and TimesFM-dependent suites will execute there); (5) paper-trade one session with DELTA_LIVE_DISPATCH disabled and read the new rejection reasons in the logs - each one is a trade the old system would have taken at degraded geometry; (6) arm live dispatch and let the guardian demonstrate the bracket patience and fill confirmation on the first positions.